

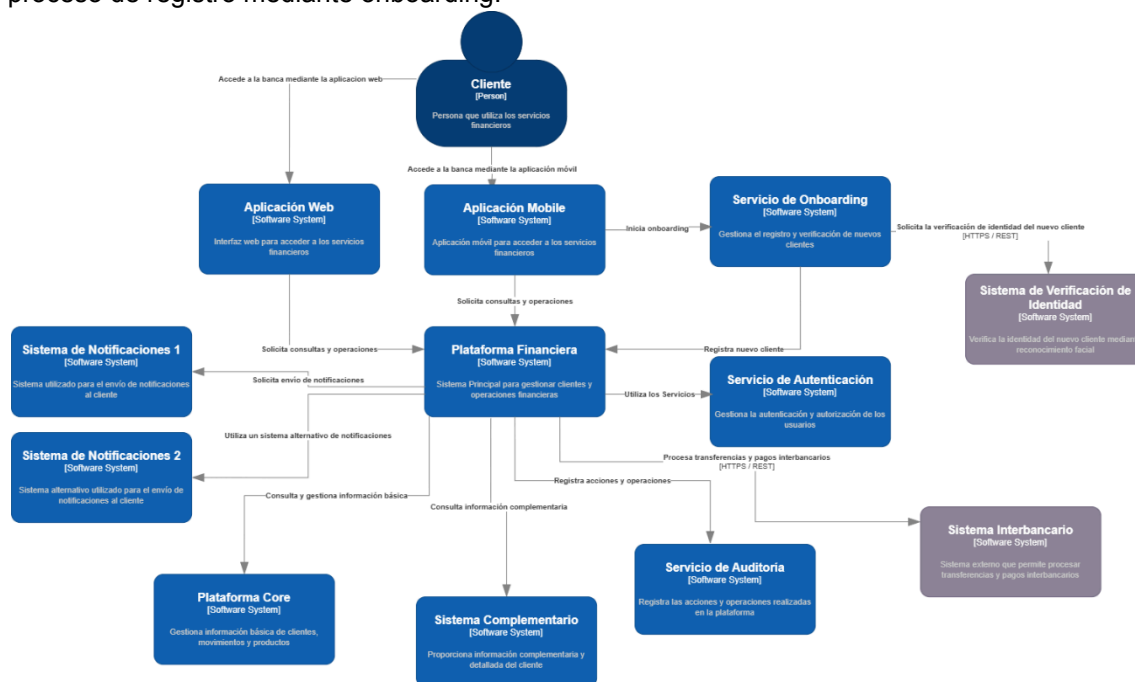
SOLUCIÓN

Se detalla la solución de la propuesta

1. DIAGRAMAS C4

1.1. Diagrama de Contexto

El diagrama de contexto demuestra una visión general de la solución de la banca internet, identificando los actores, sistemas y servicios que interactúan en la plataforma. Se describen las principales relaciones para que los clientes puedan realizar la consulta de su información, productos, revisar movimientos, realizar transferencias y pagos, autenticarse y completar el proceso de registro mediante onboarding.



Actores

- **Cliente**
Persona que utiliza los servicios de banca por internet mediante la aplicación web o móvil.

Sistemas principales

- **Aplicación Web**
Permite al cliente acceder a los servicios financieros mediante un navegador.
- **Aplicación Mobile**
Permite al cliente acceder a los servicios financieros desde un dispositivo móvil y realizar el proceso de onboarding.
- **Plataforma Financiera**
Sistema central de la solución que coordina las operaciones y servicios de banca por internet.

Sistemas relacionados

- **Servicio de Autenticación**
Gestiona la autenticación y autorización de los usuarios.
- **Plataforma Core**
Proporciona información básica de clientes, movimientos y productos.
- **Sistema Complementario**
Proporciona información adicional y detallada del cliente cuando es necesaria.
- **Servicio de Auditoría**
Registra las acciones y operaciones realizadas por los clientes.
- **Sistema de Notificaciones 1**
Permite enviar notificaciones relacionadas con las operaciones realizadas.

- Sistema de Notificaciones 2
Proporciona un mecanismo alternativo para el envío de notificaciones.
- Sistema Interbancario
Permite procesar operaciones que involucren transferencias y pagos interbancarios.
- Onboarding
Gestiona el registro de nuevos clientes y su proceso de verificación.
- Sistema de Verificación de Identidad
Permite verificar la identidad de nuevos clientes mediante reconocimiento facial.

Relaciones principales

- El Cliente accede mediante la Aplicación Web o la Aplicación Mobile.
- Las aplicaciones solicitan servicios a la Plataforma Financiera.
- La Plataforma utiliza el Servicio de Autenticación para autenticar y autorizar usuarios.
- La Plataforma consulta información de la Plataforma Core y del Sistema Complementario.
- La Plataforma registra las acciones realizadas mediante el Servicio de Auditoría.
- La Plataforma utiliza los sistemas de notificaciones para informar al cliente sobre sus movimientos.
- La Plataforma utiliza el Sistema Interbancario para las operaciones interbancarias.
- La aplicación móvil inicia el Onboarding de nuevos clientes.
- El Onboarding utiliza el Sistema de Verificación de Identidad para realizar el reconocimiento facial.

1.2. Diagrama de Contenedores

El diagrama de contenedores describe la estructura interna de la plataforma de banca por internet. En este diagrama se detallan las aplicaciones, servicios, mecanismos de almacenamiento, mensajería y las principales comunicaciones entre los sistemas existentes o externos, permitiendo observar cómo se procesan las operaciones de los clientes, cómo se ejecutan las operaciones financieras y cómo se gestionan los procesos complementarios como la autenticación, auditoría, notificaciones y onboarding.

La plataforma cuenta con dos aplicaciones cliente: una aplicación web y una aplicación móvil. Para la cual ambas utilizan el API Gateway como punto de entrada hacia los servicios internos. El API Gateway centraliza el acceso y permite aplicar políticas de seguridad y enrutamiento antes de enviar las solicitudes a los servicios correspondientes para realizar dichas transacciones.

La autenticación de los usuarios se gestiona mediante Keycloak, utilizando OAuth 2.0 y OpenID Connect. De esta manera, las aplicaciones cliente pueden autenticarse sin implementar directamente la gestión de credenciales y tokens.

Las operaciones de negocio se separan en diferentes servicios. El Servicio de Datos Básicos consulta la información de clientes y productos en la Plataforma Core y, cuando es necesario, obtener información detallada desde el Sistema Complementario. Para reducir consultas repetitivas a los sistemas fuente, se utiliza una caché Redis para almacenar temporalmente información de consulta frecuente.

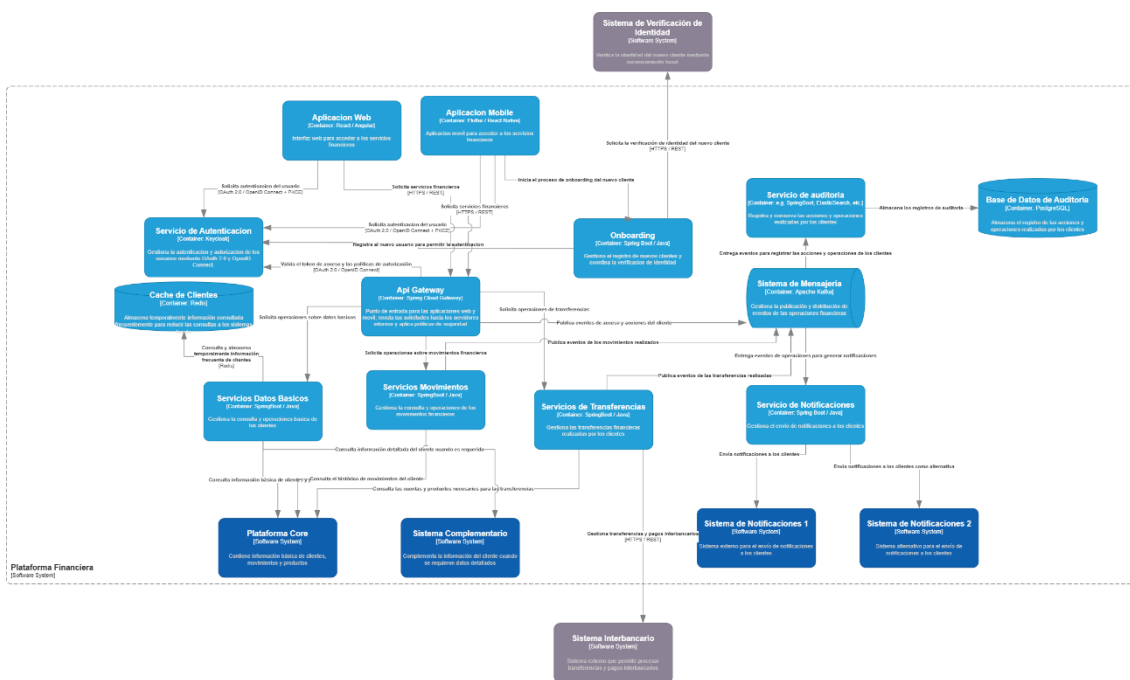
El Servicio de Movimientos permite consultar el histórico de movimientos financieros y publica eventos relacionados con las operaciones realizadas. El Servicio de Transferencias gestiona las transferencias financieras, consulta la información necesaria de las cuentas y se comunica con el Sistema Interbancario para procesar transferencias y pagos interbancarios.

La arquitectura incorpora Apache Kafka como mecanismo de mensajería para desacoplar los servicios productores de los procesos que consumen los eventos. Los eventos generados por movimientos y transferencias pueden ser utilizados por el servicio de auditoría y por el servicio de notificaciones sin generar una dependencia directa entre estos componentes.

El Servicio de Auditoría consume los eventos de las operaciones y almacena los registros en una Base de Datos de Auditoría, permitiendo conservar evidencia de las acciones realizadas por los clientes.

El Servicio de Notificaciones consume los eventos de las operaciones y se encarga de enviar las notificaciones correspondientes. Se contemplan dos sistemas de notificación, permitiendo utilizar un proveedor alternativo cuando el sistema principal no se encuentre disponible.

Esta distribución permite mantener los servicios desacoplados y con responsabilidades específicas, facilitando su mantenimiento, escalabilidad y evolución. Además, la utilización de mensajería, caché, autenticación centralizada y mecanismos alternativos de notificación contribuye a mejorar la disponibilidad y la capacidad de respuesta de la plataforma.



El diagrama de componentes presenta un mayor nivel de detalle de la Plataforma Financiera, mostrando cómo se estructura y organizan internamente los principales contenedores y las responsabilidades de sus componentes.

La autenticación y autorización se gestionan mediante componentes de seguridad integrados con **Keycloak**, utilizando **OAuth 2.0** y **OpenID Connect**. Los filtros de autenticación se encargan de validar los tokens recibidos y los componentes de autorización verifican que el usuario tenga los permisos necesarios para poder acceder a cada operación.

El servicio de movimientos utiliza un controlador y un componente de servicio para procesar las consultas de movimientos. Cuando se producen operaciones relevantes, el componente publicador genera eventos que son enviados a **Apache Kafka**.

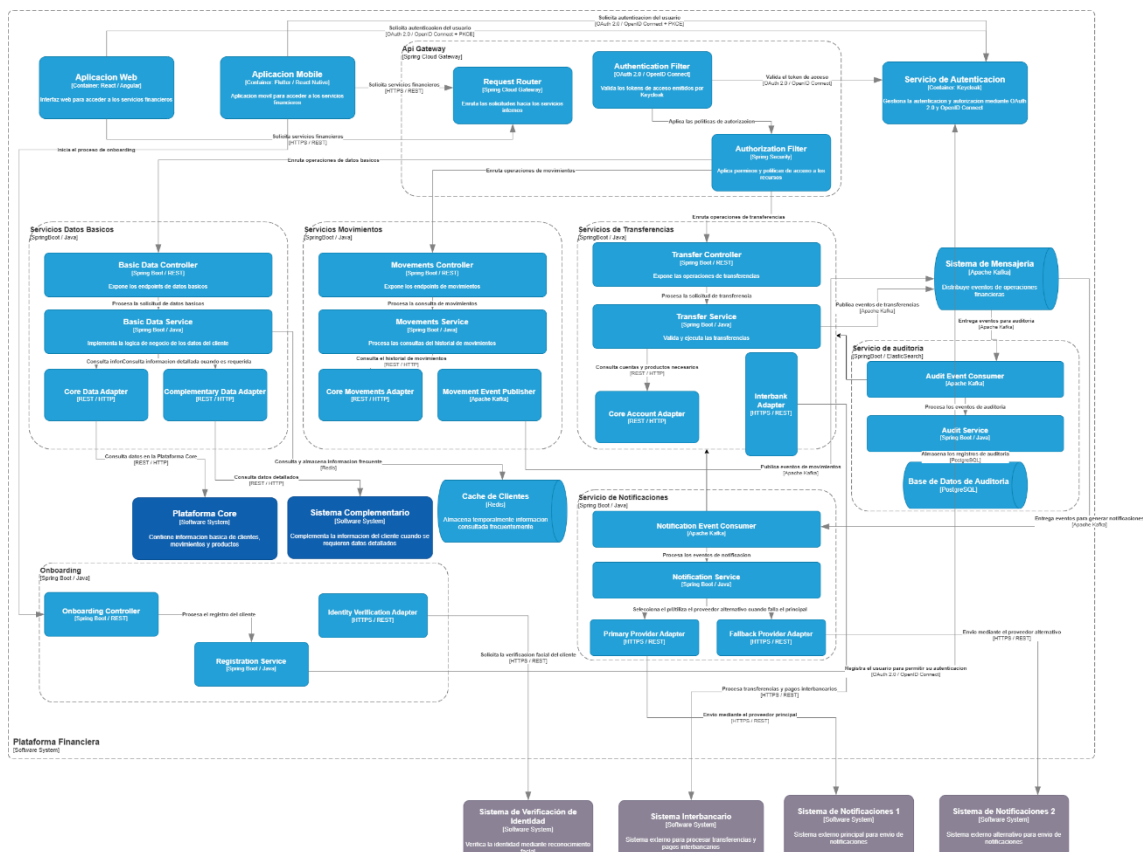
El servicio de transferencias sigue una estructura similar. El componente de servicio coordina la operación y utiliza adaptadores específicos para consultar la información de las cuentas en la Plataforma Core y comunicarse con el Sistema Interbancario. Una vez realizada una operación, se publica un evento en Kafka para que otros procesos puedan utilizarlo.

La mensajería mediante Kafka permite desacoplar el procesamiento de las operaciones de funciones complementarias como la auditoría y las notificaciones. El servicio de auditoría consume estos eventos, procesa la información y almacena los registros en la Base de Datos de Auditoría.

El servicio de notificaciones también consume los eventos publicados en Kafka. Su componente de servicio determina el envío y utiliza adaptadores independientes para comunicarse con el sistema de notificaciones principal o con el sistema alternativo. Esta separación nos permite incorporar un mecanismo de alta disponibilidad al poder reemplazar proveedores sin modificar directamente la lógica principal del servicio.

El proceso de onboarding se divide en componentes encargados de recibir la solicitud, registrar al nuevo cliente y comunicarse con el sistema externo de verificación de identidad mediante reconocimiento facial. Una vez completado el registro, el servicio de registro se integra con Keycloak para permitir posteriormente la autenticación del nuevo usuario.

La separación de componentes permite aplicar responsabilidades específicas dentro de cada servicio y mantener bajo acoplamiento entre las diferentes partes de la solución. Los adaptadores también permiten aislar las dependencias con sistemas externos, facilitando futuras modificaciones o incorporación de nuevos proveedores.



2. Arquitectura de la Plataforma

Se decidió utilizar una arquitectura basada en microservicios y eventos para organizar las principales capacidades de la Plataforma Financiera. Las responsabilidades se distribuyen en servicios independientes, como Datos Básicos, Movimientos, Transferencias, Onboarding, Auditoría y Notificaciones.

Así también, se decidió utilizar una arquitectura orientada a eventos para desacoplar funcionalidades complementarias que no deberían afectar el procesamiento de las operaciones principales, como la auditoría y las notificaciones. Para ello, los servicios principales publican eventos de las operaciones realizadas mediante el sistema de mensajería Apache Kafka, permitiendo que otros servicios los procesen de forma independiente.

Esta decisión busca mantener una separación clara de responsabilidades y reducir el acoplamiento entre las diferentes funcionalidades del sistema. Cada servicio se concentra en una responsabilidad específica y puede evolucionar de manera independiente, siempre que mantenga los contratos de comunicación definidos con los demás componentes.

Justificación

1. Separación de responsabilidades

La división de la solución en servicios permite aplicar el principio de responsabilidad única, evitando concentrar toda la lógica de negocio en una única aplicación. Por ejemplo, el servicio de Transferencias puede procesar una operación sin depender directamente del funcionamiento interno del servicio de Auditoría o del servicio de Notificaciones.

Esto facilita el mantenimiento, las pruebas y la evolución de cada funcionalidad.

2. Desacoplamiento mediante eventos

La utilización de eventos permite separar el procesamiento de las operaciones financieras de funcionalidades complementarias. Una vez realizada una operación, el servicio correspondiente puede publicar un evento en Kafka para que los servicios de Auditoría y Notificaciones lo procesen de manera independiente.

De esta manera, un inconveniente temporal en el procesamiento de una notificación o en la auditoría no tiene por qué interrumpir una operación financiera que ya fue procesada correctamente. Además, los eventos pueden permanecer disponibles para ser procesados posteriormente, dependiendo de la configuración del sistema de mensajería.

3. Escalabilidad independiente

Las diferentes funcionalidades pueden presentar necesidades de procesamiento distintas. Por ejemplo, las consultas de movimientos pueden requerir mayor capacidad durante determinados períodos, mientras que otros servicios pueden mantener una carga menor.

Con microservicios es posible escalar individualmente aquellos servicios que lo necesiten, evitando tener que incrementar los recursos de toda la aplicación.

Alternativa evaluada: microservicios con comunicación principalmente síncrona

Como alternativa se consideró implementar los microservicios utilizando principalmente comunicación síncrona, donde un servicio realiza una solicitud directa a otro servicio para completar funcionalidades complementarias como auditoría o notificaciones.

Aunque este enfoque puede ser sencillo de implementar y comprender inicialmente, genera un mayor acoplamiento entre los servicios. Por ejemplo, si una operación financiera dependiera directamente de que el servicio de auditoría o el servicio de notificaciones estuviera disponible para finalizar su procesamiento, una falla temporal en alguno de estos servicios podría afectar una operación que ya se encuentra en proceso.

Por esta razón, se adopta una combinación de microservicios y arquitectura orientada a eventos, utilizando Kafka para desacoplar procesos complementarios de las operaciones principales. Esta decisión permite mantener las responsabilidades separadas y mejorar la tolerancia ante fallos, manteniendo una arquitectura preparada para incorporar nuevas funcionalidades.

Este enfoque se encuentra representado en los diagramas C4 mediante los servicios independientes y el sistema de mensajería Apache Kafka, que distribuye los eventos hacia los servicios de Auditoría y Notificaciones.

3. Componentes de la Plataforma

3.1. Api Gateway

Para implementar el API Gateway requerido por la arquitectura, se seleccionó Spring Cloud Gateway como tecnología para gestionar el acceso de las aplicaciones Web y Mobile hacia los servicios internos.

Justificación

1. Integración con Spring Boot y Java

Spring Cloud Gateway forma parte del ecosistema Spring y está diseñado para integrarse con aplicaciones desarrolladas con Spring Boot. Debido a que los servicios principales de la plataforma utilizan Java y Spring Boot, se mantiene una mayor homogeneidad tecnológica entre el Gateway y los microservicios.

Esto facilita la configuración, desarrollo y mantenimiento de la solución, ya que el equipo puede utilizar tecnologías y mecanismos conocidos dentro del mismo ecosistema.

2. Integración con la arquitectura de seguridad

La plataforma utiliza OAuth 2.0 y OpenID Connect junto con Keycloak como proveedor de identidad. Spring Cloud Gateway puede integrarse con el ecosistema de seguridad de Spring y con estos mecanismos de autenticación y autorización.

Esto permite mantener una implementación coherente entre el Gateway y los servicios desarrollados con Spring Boot.

Alternativa evaluada: Amazon API Gateway

Como alternativa se evaluó Amazon API Gateway, servicio administrado de AWS utilizado para crear, publicar, proteger, supervisar y gestionar APIs.

Entre sus principales ventajas se encuentran:

- Es un servicio administrado, por lo que AWS se encarga de gran parte de la infraestructura necesaria para su operación.
- Se integra directamente con otros servicios de AWS, facilitando la construcción de arquitecturas basadas en servicios cloud.
- Proporciona funcionalidades para seguridad, control de tráfico, monitoreo y gestión de APIs.
- Permite escalar la infraestructura administrada por AWS sin tener que gestionar directamente los servidores donde se ejecuta el Gateway.

Sin embargo, también presenta algunas desventajas para la solución planteada:

- Genera una mayor dependencia del ecosistema de AWS (vendor lock-in).
- Su utilización introduce una tecnología adicional diferente al ecosistema Java/Spring Boot utilizado por los microservicios, por ende la hace dependiente a esta nube.
- El costo depende del volumen de solicitudes y de las características utilizadas, por lo que debe ser considerado dentro del presupuesto operativo de la plataforma.
- Algunas configuraciones y funcionalidades quedan condicionadas por las características y servicios proporcionados por AWS.

Selección final

Después de evaluar ambas alternativas, se selecciona Spring Cloud Gateway debido principalmente a su integración con el ecosistema Java y Spring Boot definido para los microservicios de la Plataforma Financiera y a su vez nos proporciona mayor portabilidad al sistema cuando queremos cambiarnos de proveedor de nube.

Amazon API Gateway permanece como una alternativa viable, especialmente si la infraestructura se orientara fuertemente hacia AWS y se priorizará el uso de servicios administrados. Sin embargo, para la arquitectura propuesta se prioriza mantener una mayor coherencia tecnológica y control sobre la implementación del Gateway.

3.2. Framework para las aplicaciones Web y Mobile

Para el desarrollo de las aplicaciones cliente se deben utilizar dos interfaces independientes: una aplicación Web tipo SPA y una aplicación Mobile multiplataforma.

Para la aplicación Web se seleccionó **React**, mientras que para la aplicación Mobile se seleccionó **Flutter**.

La selección se realizó considerando factores como mantenibilidad, rendimiento, reutilización de componentes, facilidad de desarrollo y capacidad para construir aplicaciones que puedan evolucionar de acuerdo con las necesidades de la plataforma financiera.

3.2.1. Aplicación Web — React

Se seleccionó **React** como framework/biblioteca principal para el desarrollo de la aplicación Web tipo SPA.

Justificación

1. Arquitectura basada en componentes

React permite construir la interfaz mediante componentes reutilizables. Esto facilita separar funcionalidades de la aplicación, como autenticación, consulta de movimientos, transferencias y visualización de información del cliente.

Esta característica favorece la mantenibilidad y permite reutilizar componentes a medida que la aplicación incorpora nuevas funcionalidades.

2. Adecuación para aplicaciones SPA

React permite desarrollar aplicaciones de una sola página (SPA), donde gran parte de la interacción ocurre en el cliente sin necesidad de recargar completamente la aplicación en cada operación.

Esto permite construir una interfaz adecuada para una plataforma de banca por internet que realiza múltiples consultas y operaciones contra los servicios expuestos por el API Gateway.

Alternativa evaluada: Angular

Como alternativa se evaluó Angular, que proporciona un framework completo para el desarrollo de aplicaciones Web.

Entre sus ventajas se encuentran una estructura más definida para proyectos grandes, funcionalidades integradas para diferentes necesidades del desarrollo y una arquitectura que facilita establecer convenciones comunes dentro del proyecto.

Como desventajas frente a React, puede presentar una mayor complejidad inicial y una curva de aprendizaje más amplia debido a la cantidad de conceptos y funcionalidades que incorpora. Se selecciona React debido a su modelo basado en componentes, flexibilidad y adecuación para construir la SPA propuesta.

3.2.2. Aplicación Mobile — Flutter

Para la aplicación Mobile se seleccionó **Flutter** como framework multiplataforma.

Justificación

1. Desarrollo multiplataforma

Flutter permite desarrollar una aplicación utilizando una única base de código para diferentes plataformas móviles. Esto reduce la necesidad de mantener implementaciones completamente independientes para cada sistema operativo.

Para la plataforma financiera esto facilita la evolución y mantenimiento de la aplicación Mobile.

2. Rendimiento y experiencia de usuario

Flutter utiliza su propio motor de renderizado y permite construir interfaces con un comportamiento consistente entre las diferentes plataformas soportadas.

Esto resulta adecuado para una aplicación financiera que requiere una interfaz fluida y consistente para realizar consultas, transferencias, pagos y procesos de autenticación.

Alternativa evaluada: React Native

Como alternativa se evaluó React Native, que también permite desarrollar aplicaciones móviles multiplataforma utilizando una base de código compartida y tecnologías relacionadas con JavaScript/TypeScript.

Entre sus ventajas se encuentra la posibilidad de reutilizar conocimientos del ecosistema React y una amplia disponibilidad de librerías y herramientas.

Como desventaja, determinadas funcionalidades pueden requerir integración con código nativo dependiendo de las características necesarias de la aplicación y de las plataformas objetivo.

Se selecciona Flutter debido a que proporciona una solución multiplataforma con una base de código común y un control consistente sobre la interfaz de usuario y el comportamiento visual de la aplicación.

3.2.3. Arquitectura de Autenticación y autorización

Para implementar el mecanismo de autenticación y autorización requerido por la plataforma se seleccionó **Keycloak** como proveedor de identidad, utilizando los estándares **OAuth 2.0** y **OpenID Connect (OIDC)**.

La selección de Keycloak se realizó considerando la necesidad de contar con una solución especializada en gestión de identidades que pueda integrarse con las aplicaciones Web y Mobile, así como con los servicios internos de la plataforma.

Para las aplicaciones Web y Mobile se utilizará el flujo Authorization Code con PKCE, adecuado para clientes que no deben almacenar directamente credenciales sensibles y que requieren obtener tokens de acceso de forma segura.

Justificación

1. Gestión centralizada de identidad

Keycloak permite centralizar la gestión de usuarios, autenticación, roles y permisos en un único componente especializado.

Esto evita que cada microservicio tenga que implementar su propio mecanismo de autenticación y permite mantener una política de seguridad uniforme para las diferentes aplicaciones y servicios.

2. Compatibilidad con estándares de seguridad

Keycloak proporciona soporte para estándares como OAuth 2.0 y OpenID Connect, que son los protocolos definidos para la autenticación y autorización de la plataforma.

Esto permite que las aplicaciones Web y Mobile puedan autenticarse mediante un mecanismo estándar y que los servicios internos puedan validar los tokens emitidos por el proveedor de identidad.

Alternativa evaluada: Amazon Cognito

Como alternativa se evaluó Amazon Cognito, servicio administrado de AWS para gestión de identidad y autenticación de usuarios.

Entre sus ventajas se encuentran la integración directa con otros servicios de AWS, la administración de la infraestructura por parte del proveedor y la disponibilidad de funcionalidades relacionadas con autenticación y gestión de usuarios.

Como desventajas, introduce una mayor dependencia del ecosistema AWS y reduce el control directo sobre determinados aspectos de la infraestructura y configuración del proveedor de identidad.

Se selecciona Keycloak debido a que proporciona una solución especializada de gestión de identidad que puede desplegarse dentro de la infraestructura elegida para la plataforma, ofreciendo mayor control sobre su configuración y manteniendo independencia respecto de un proveedor cloud específico.

Flujo de autenticación

Para la aplicación Web y la aplicación Mobile se utilizará el flujo Authorization Code con PKCE.

De forma general, el proceso será:

1. El usuario inicia sesión desde la aplicación Web o Mobile.
2. La aplicación redirige al usuario hacia Keycloak.
3. Keycloak realiza la autenticación del usuario.
4. Keycloak devuelve un código de autorización.
5. La aplicación utiliza PKCE para intercambiar el código por los tokens correspondientes.
6. La aplicación utiliza el Access Token para realizar solicitudes al API Gateway.
7. El API Gateway valida el token y las políticas de autorización antes de permitir el acceso al servicio correspondiente.

De esta forma, las credenciales del usuario permanecen bajo la responsabilidad del proveedor de identidad y los microservicios reciben únicamente los tokens necesarios para autorizar las operaciones.

3.2.4.Arquitectura de Onboarding y verificación de identidad

Para el proceso de registro de nuevos clientes se definió un servicio de **Onboarding** encargado de coordinar el registro del usuario y la verificación de su identidad.

La aplicación Mobile inicia el proceso y se comunica con el servicio de Onboarding mediante HTTPS. Este servicio coordina la verificación de identidad mediante reconocimiento facial utilizando un Identity Verification Adapter, que permite integrar un proveedor especializado sin acoplar directamente la lógica de negocio al proveedor seleccionado.

Una vez realizada la verificación, el resultado es utilizado por el servicio de Onboarding para continuar con el proceso de registro del usuario y su posterior habilitación para autenticarse mediante Keycloak.

La arquitectura propuesta separa la verificación de identidad del mecanismo de autenticación. El reconocimiento facial se utiliza durante el onboarding para comprobar la identidad del nuevo cliente, mientras que la autenticación posterior se realiza mediante el mecanismo definido para los usuarios de la plataforma.

Justificación

1. Utilización de un servicio especializado de verificación de identidad

Se decidió utilizar un servicio especializado para la verificación facial en lugar de desarrollar internamente toda la tecnología de reconocimiento facial.

Desarrollar esta capacidad desde cero implicaría implementar y mantener algoritmos de reconocimiento, procesamiento de imágenes, mecanismos de detección de identidad y controles de seguridad especializados. Utilizar un proveedor especializado permite reducir esta complejidad y concentrar el desarrollo interno en la lógica propia del negocio financiero.

2. Uso del patrón Adapter

La integración con el proveedor de reconocimiento facial se realiza mediante un **Identity Verification Adapter**.

Este componente permite separar el servicio de Onboarding de la implementación específica del proveedor. De esta manera, si posteriormente se cambia de proveedor de verificación de identidad, el cambio puede concentrarse principalmente en el Adapter sin modificar la lógica principal del proceso de onboarding.

Esta decisión reduce el acoplamiento y facilita la evolución futura de la solución.

3. Separación entre verificación de identidad y autenticación

La verificación facial se mantiene como parte del proceso de onboarding y no como mecanismo principal de autenticación de la banca digital.

Esta separación permite que el proveedor de identidad, Keycloak, continúe siendo responsable de la autenticación y autorización de los usuarios, mientras que el servicio de Onboarding se concentra en el proceso de registro y verificación inicial.

Alternativa evaluada: AWS Rekognition

Como alternativa tecnológica se evaluó AWS Rekognition, servicio de AWS que proporciona capacidades de análisis y reconocimiento de imágenes y rostros.

Entre sus principales ventajas se encuentran:

- Servicio administrado, reduciendo la necesidad de administrar infraestructura propia para el procesamiento.
- Integración con otros servicios de AWS.
- Capacidad de procesamiento escalable.
- Permite incorporar capacidades de análisis facial sin desarrollar los algoritmos desde cero.

Entre sus desventajas se encuentran:

- Genera dependencia del ecosistema de AWS.
- El costo depende del volumen de procesamiento y de las funcionalidades utilizadas.
- La solución debe considerar los requisitos regulatorios y de protección de datos aplicables al tratamiento de información biométrica.
- La integración queda condicionada a las capacidades y características proporcionadas por AWS.

Por estas razones, se mantiene el Identity Verification Adapter como punto de integración. De esta forma, AWS Rekognition puede ser utilizado como una alternativa de implementación sin que el servicio de Onboarding quede directamente acoplado a AWS.

Flujo del proceso

El flujo definido para el onboarding es:

1. El nuevo cliente inicia el registro desde la aplicación Mobile.
2. Mobile envía la solicitud al servicio de Onboarding mediante HTTPS.
3. Onboarding recopila y valida la información necesaria para el registro.
4. El Registration Service solicita la verificación de identidad al Identity Verification Adapter.
5. El Adapter se comunica con el proveedor especializado de verificación facial.
6. El proveedor devuelve el resultado de la verificación.
7. Si la identidad es validada, Onboarding continúa con el registro del usuario.
8. El usuario queda preparado para utilizar el mecanismo de autenticación gestionado por Keycloak.

De esta manera, el proceso de onboarding queda desacoplado del proveedor específico de reconocimiento facial y puede evolucionar sin modificar la lógica principal de la plataforma.

3.2.5.Arquitectura de acceso a datos y selección de tecnología de caché

Para atender el requisito de almacenar temporalmente información consultada frecuentemente por los clientes, se seleccionó **Redis** como tecnología de caché.

Redis se utilizará como una capa de almacenamiento temporal entre los servicios de la plataforma y los sistemas fuente, principalmente para información que sea consultada con frecuencia y que no requiera ser obtenida nuevamente en cada solicitud.

Justificación

1. Alto rendimiento y baja latencia

Redis mantiene los datos principalmente en memoria, permitiendo realizar operaciones de lectura y escritura con una latencia muy baja.

Esta característica es adecuada para una plataforma financiera donde se busca proporcionar respuestas rápidas a consultas frecuentes, reduciendo el tiempo necesario para obtener información desde los sistemas fuente.

2. Compatibilidad con una arquitectura distribuida

Redis puede utilizarse como una caché compartida entre diferentes instancias de los servicios. Esto resulta importante cuando los microservicios se escalan horizontalmente, ya que las diferentes instancias pueden acceder a una misma capa de caché.

Alternativa evaluada: Caffeine

Como alternativa se evaluó Caffeine, una biblioteca de caché para aplicaciones Java.

Entre sus ventajas se encuentran:

- Excelente rendimiento al almacenar datos directamente en memoria.
- Integración sencilla con aplicaciones Java y Spring Boot.
- No requiere administrar un servicio externo para utilizar una caché local.
- Es adecuada para información que solamente necesita ser utilizada por una instancia específica de la aplicación.

Sin embargo, presenta como desventaja que la caché se encuentra asociada a la instancia de la aplicación. En un escenario donde existen múltiples instancias de un microservicio, cada instancia podría mantener una caché diferente.

Esto puede generar duplicación de información y comportamientos diferentes entre las instancias.

Por esta razón, se selecciona Redis, ya que permite disponer de una caché compartida y desacoplada de las instancias individuales de los microservicios, facilitando el escalamiento horizontal de la plataforma.

3.2.6.Arquitectura de acceso a sistemas existentes

Para acceder a la información proporcionada por la **Plataforma Core** y el **Sistema Complementario**, se decidió utilizar el patrón arquitectónico **Adapter** como mecanismo de integración entre los microservicios de la Plataforma Financiera y los sistemas existentes.

Cada servicio utiliza un adaptador específico para comunicarse con el sistema que proporciona la información requerida. De esta manera, la lógica de negocio de los microservicios no depende directamente de los protocolos, estructuras de datos o mecanismos de comunicación particulares de los sistemas existentes.

En la solución se definen adaptadores como Core Data Adapter, Core Movements Adapter y Core Account Adapter, además del Complementary Data Adapter.

Justificación

1. Desacoplamiento de los sistemas existentes

Los sistemas Core y Complementario pueden utilizar tecnologías, estructuras de datos o interfaces diferentes a las utilizadas por los nuevos microservicios.

El patrón Adapter permite encapsular estas diferencias dentro de componentes específicos, evitando que la lógica de negocio de los servicios tenga que conocer los detalles internos de cada sistema.

Por ejemplo, el servicio de Datos Básicos puede solicitar información mediante una interfaz propia, mientras que el Core Data Adapter se encarga de transformar esa solicitud al formato requerido por el Core.

2. Facilidad para incorporar o reemplazar sistemas

El uso de adaptadores facilita la evolución de la arquitectura. Si en el futuro se reemplaza un sistema existente o cambia la forma en que se expone su información, el cambio puede concentrarse principalmente en el adaptador correspondiente.

Esto reduce el impacto sobre los servicios de negocio y permite mantener estable la interfaz utilizada internamente por los microservicios.

3. Separación entre lógica de negocio e integración

Los servicios de negocio deben concentrarse en las reglas correspondientes a su responsabilidad, mientras que los adaptadores se encargan de la comunicación con sistemas externos o existentes.

Esta separación facilita las pruebas, el mantenimiento y la reutilización de los componentes de integración.

Alternativa evaluada: integración directa desde los servicios

Como alternativa se evaluó permitir que cada servicio de negocio se comunique directamente con los sistemas Core y Complementario.

Entre sus ventajas se encuentra una implementación inicialmente más sencilla, ya que no sería necesario incorporar componentes adicionales de adaptación.

Sin embargo, esta alternativa genera un mayor acoplamiento entre la lógica de negocio y los sistemas existentes. Los servicios tendrían que conocer directamente sus APIs, protocolos, formatos de datos y particularidades de integración.

Además, un cambio en alguno de los sistemas fuente podría requerir modificaciones en varios servicios.

Por esta razón, se selecciona el patrón Adapter, ya que permite encapsular las diferencias de integración y mantener desacoplada la lógica de negocio de los mecanismos específicos utilizados por los sistemas existentes.

Aplicación en la solución

El patrón se aplica de la siguiente manera:

- Basic Data Service → Core Data Adapter → Plataforma Core
- Basic Data Service → Complementary Data Adapter → Sistema Complementario
- Movements Service → Core Movements Adapter → Plataforma Core
- Transfer Service → Core Account Adapter → Plataforma Core
- Transfer Service → Interbank Adapter → Sistema Interbancario

Esta estructura permite que los servicios de negocio trabajen mediante interfaces internas estables, mientras los adaptadores encapsulan los detalles específicos de cada sistema integrado.

3.2.7.Arquitectura de eventos y mensajería

Para implementar la comunicación orientada a eventos definida en la arquitectura, se seleccionó **Apache Kafka** como plataforma de mensajería y distribución de eventos.

Kafka será utilizado principalmente para distribuir eventos generados por las operaciones financieras hacia servicios que requieren procesarlos de manera independiente, como **Auditoría y Notificaciones**.

Justificación

1. Desacoplamiento entre productores y consumidores

Kafka permite que los servicios que generan eventos publiquen la información sin tener que comunicarse directamente con cada uno de los servicios consumidores.

Por ejemplo, el servicio de Transferencias puede publicar un evento relacionado con una transferencia realizada, mientras que Auditoría y Notificaciones pueden consumir ese evento de manera independiente.

Esto reduce el acoplamiento entre los servicios y facilita incorporar nuevos consumidores en el futuro.

2. Persistencia y procesamiento de eventos

Kafka permite mantener los eventos durante un período configurado, permitiendo que los consumidores puedan procesarlos posteriormente.

Esta característica es importante para funcionalidades como auditoría y notificaciones, ya que un consumidor que presente una interrupción temporal puede continuar procesando los eventos pendientes cuando vuelva a estar disponible.

3. Escalabilidad

Kafka está diseñado para manejar grandes volúmenes de eventos mediante mecanismos de partición y distribución.

Esto permite que la plataforma pueda incrementar la capacidad de procesamiento de los consumidores conforme aumente el número de operaciones realizadas por los clientes.

Alternativa evaluada: RabbitMQ

Como alternativa se evaluó RabbitMQ, una plataforma de mensajería ampliamente utilizada para implementar comunicación asíncrona entre aplicaciones.

Entre sus ventajas se encuentran:

- Modelo de colas y enrutamiento flexible.
- Administración y configuración relativamente sencilla.
- Adecuado para la distribución de mensajes entre productores y consumidores.
- Buen soporte para patrones tradicionales de mensajería.

Como desventaja frente a Kafka para nuestra arquitectura, RabbitMQ está más orientado al modelo tradicional de broker y colas de mensajes, mientras que Kafka está especialmente diseñado para manejar flujos de eventos y mantenerlos disponibles para múltiples consumidores.

Para la Plataforma Financiera se selecciona Apache Kafka debido a que el diseño requiere distribuir eventos de operaciones hacia diferentes consumidores, como Auditoría y Notificaciones, y mantener una arquitectura orientada a eventos que pueda incorporar nuevos consumidores en el futuro.

Aplicación en la solución

El flujo principal de eventos será:

Movimientos → Kafka → Auditoría / Notificaciones

Transferencias → Kafka → Auditoría / Notificaciones

De esta manera, los servicios de negocio publican los eventos y los consumidores los procesan de forma independiente, manteniendo desacopladas las operaciones principales de los procesos complementarios.

3.2.8.Arquitectura y tecnología para auditoría

Para registrar las acciones y operaciones realizadas por los clientes se definió un **servicio independiente de auditoría**, encargado de recibir y almacenar los eventos generados por los diferentes servicios de la plataforma.

Los eventos de auditoría serán distribuidos mediante Apache Kafka, permitiendo que el registro de las acciones se realice de forma desacoplada respecto de los servicios que ejecutan las operaciones principales.

Para la persistencia de los registros de auditoría se seleccionó PostgreSQL.

Justificación

1. Consistencia e integridad de los registros

PostgreSQL proporciona mecanismos de persistencia transaccional y restricciones de integridad que permiten mantener los registros de auditoría de forma estructurada.

Esto resulta importante debido a que los registros deben conservar información relacionada con las acciones realizadas por los clientes y servir como evidencia para procesos de control, investigación y cumplimiento.

2. Capacidad de consulta y trazabilidad

Los registros de auditoría pueden necesitar ser consultados posteriormente para identificar quién realizó una acción, qué operación realizó, cuándo ocurrió y cuál fue el resultado.

El modelo relacional de PostgreSQL permite estructurar esta información y utilizar consultas para realizar búsquedas y análisis sobre los registros almacenados.

3. Separación del almacenamiento de auditoría

La base de datos de auditoría se mantiene separada de las bases de datos utilizadas para la operación principal.

Esto permite que los registros de auditoría tengan un ciclo de vida y políticas de acceso independientes, reduciendo el impacto que las operaciones normales de la plataforma puedan tener sobre la información de auditoría.

Alternativa evaluada: Elasticsearch

Como alternativa se evaluó Elasticsearch, una plataforma especializada en almacenamiento y búsqueda de grandes cantidades de información, especialmente útil para registros y datos que requieren búsquedas rápidas.

Entre sus ventajas se encuentran:

- Capacidad de realizar búsquedas eficientes sobre grandes volúmenes de registros.
- Adecuado para análisis y exploración de logs y eventos.
- Integración con herramientas de observabilidad y visualización como Kibana.
- Puede ser utilizado para realizar análisis sobre grandes cantidades de información.

Como desventajas, introduce una plataforma adicional y una arquitectura de almacenamiento diferente a una base de datos relacional. Además, para los registros de auditoría que requieren una estructura consistente y persistencia transaccional, PostgreSQL proporciona un modelo relacional adecuado.

Por esta razón, se selecciona PostgreSQL como almacenamiento principal de auditoría, mientras que Elasticsearch puede considerarse posteriormente como una tecnología complementaria para análisis y búsqueda avanzada si el volumen de registros lo requiere.

Flujo de auditoría

El flujo definido es:

Servicio de Movimientos / Transferencias → Kafka → Audit Event Consumer → Audit Service → PostgreSQL

Además, otros servicios de la plataforma pueden publicar eventos de auditoría para mantener un registro centralizado de las acciones realizadas.

De esta manera, la auditoría permanece desacoplada de la lógica de negocio y dispone de un almacenamiento independiente para conservar los registros.

3.2.9.Arquitectura de notificaciones

Para cumplir con el requisito de notificar a los clientes sobre los movimientos y operaciones realizadas, se definió un Servicio de Notificaciones independiente que consume los eventos publicados en Apache Kafka.

El servicio se encarga de procesar los eventos y determinar el mecanismo mediante el cual se enviará la notificación al cliente. Para garantizar disponibilidad y continuidad del servicio, se

integran dos mecanismos de notificación: un proveedor externo especializado y una API de Notificaciones propia desarrollada por la entidad.

La comunicación con ambos mecanismos se realiza mediante componentes Adapter, permitiendo mantener desacoplado el Servicio de Notificaciones de las implementaciones específicas de cada mecanismo.

Justificación

1. Desacoplamiento mediante eventos

El Servicio de Notificaciones consume los eventos generados por las operaciones financieras desde Kafka, en lugar de que los servicios de negocio tengan que comunicarse directamente con los mecanismos de notificación.

Esto permite que una interrupción temporal del servicio de notificaciones no tenga que detener directamente el procesamiento de una operación financiera.

2. Disponibilidad mediante mecanismo alternativo

Se incorporan dos mecanismos de notificación para reducir la dependencia de un único proveedor externo.

Como mecanismo principal se utiliza **Latinia**, mientras que como alternativa se dispone de una **API de Notificaciones propia** desarrollada por la entidad.

Si el proveedor principal presenta una indisponibilidad o un error temporal, el Servicio de Notificaciones puede utilizar la API propia de acuerdo con las reglas de contingencia definidas.

Esto mejora la tolerancia ante fallos y permite mantener la capacidad de notificar al cliente ante una interrupción del proveedor principal.

3. Uso del patrón Adapter

Se utilizan un **Primary Provider Adapter** y un **Fallback Provider Adapter** para encapsular la comunicación con cada mecanismo.

El Primary Provider Adapter se encarga de la integración con Latinia, mientras que el Fallback Provider Adapter encapsula la comunicación con la API de Notificaciones propia.

De esta manera, el Servicio de Notificaciones mantiene una interfaz común y no necesita conocer los detalles específicos de cada implementación, facilitando su reemplazo o la incorporación de nuevos mecanismos en el futuro.

Alternativa evaluada: utilizar dos proveedores externos

Como alternativa se consideró utilizar dos proveedores externos de notificaciones.

Esta opción tendría como ventaja reducir el desarrollo y mantenimiento de componentes propios, además de permitir disponer de dos proveedores especializados.

Sin embargo, incrementa la dependencia de servicios externos y requeriría mantener dos integraciones, contratos y configuraciones independientes.

Se seleccionó la combinación de Latinia como proveedor externo principal y una API de Notificaciones propia como mecanismo alternativo, buscando equilibrar el uso de una solución especializada con el control interno sobre el mecanismo de contingencia.

Flujo de notificaciones

Movimientos / Transferencias → Kafka → Notification Event Consumer → Servicio de Notificaciones → Primary Provider Adapter → Latinia

En caso de indisponibilidad del proveedor principal:

Servicio de Notificaciones → Fallback Provider Adapter → API de Notificaciones propia

3.2.10. Alta disponibilidad, tolerancia a fallos y recuperación ante desastres

Para implementar los requisitos de **alta disponibilidad, tolerancia a fallos, recuperación ante desastres y auto-healing**, se propone utilizar **Amazon Elastic Kubernetes Service (Amazon EKS)** como plataforma para ejecutar los contenedores de los diferentes servicios de la arquitectura.

Los microservicios se desplegarán mediante múltiples réplicas distribuidas entre diferentes zonas de disponibilidad. Kubernetes permitirá administrar el estado de los contenedores, detectar fallos y crear nuevas instancias cuando sea necesario.

Justificación

1. Auto-healing y recuperación automática

Kubernetes permite definir el estado esperado de cada servicio mediante sus configuraciones de despliegue. Si una instancia de un microservicio falla, el orquestador puede detectar la situación y crear automáticamente una nueva instancia para mantener la cantidad de réplicas configurada.

Esto permite reducir la intervención manual ante fallos de los contenedores y mantener la continuidad de los servicios.

2. Distribución entre zonas de disponibilidad

Amazon EKS permite distribuir las cargas de trabajo entre diferentes zonas de disponibilidad dentro de una región.

De esta manera, la arquitectura no depende de una única zona. Si una zona presenta una interrupción, las réplicas ubicadas en otras zonas pueden continuar atendiendo las solicitudes.

3. Escalabilidad horizontal

Kubernetes permite aumentar o reducir dinámicamente el número de réplicas de los servicios de acuerdo con la demanda.

Esto resulta adecuado para una plataforma financiera, ya que determinados servicios, como consultas de movimientos o datos básicos, pueden experimentar diferentes niveles de carga durante el día.

4. Recuperación ante desastres

La arquitectura debe complementarse con mecanismos de respaldo y recuperación de los datos. Las bases de datos y demás componentes persistentes deberán utilizar mecanismos de backup y replicación que permitan recuperar la información ante una falla grave.

Para escenarios de desastre que afecten a una región completa, se puede mantener una estrategia de recuperación en una región secundaria, de acuerdo con los objetivos de **RTO (Recovery Time Objective)** y **RPO (Recovery Point Objective)** definidos por la entidad.

Alternativa evaluada: Amazon ECS

Como alternativa se consideró **Amazon Elastic Container Service (Amazon ECS)**.

ECS permite ejecutar y administrar contenedores en AWS y presenta una configuración más sencilla que Kubernetes. Además, al ser un servicio administrado, reduce parte de la complejidad operacional relacionada con la administración del orquestador.

Sin embargo, Kubernetes proporciona una plataforma más flexible para administrar una arquitectura compuesta por múltiples microservicios, además de contar con un ecosistema amplio de herramientas para despliegue, escalabilidad, observabilidad y recuperación automática.

Por este motivo, se selecciona **Amazon EKS** como plataforma de orquestación de los contenedores.

Aplicación en la arquitectura

La infraestructura se organizará de forma que los principales servicios puedan ejecutarse como múltiples réplicas:

API Gateway → Microservicios → Servicios de soporte

Las réplicas de los servicios se distribuirán entre diferentes zonas de disponibilidad y Kubernetes administra su estado.

Ante el fallo de una instancia:

Fallo del contenedor → Kubernetes detecta el fallo → se crea una nueva réplica → el servicio vuelve a estar disponible.

Para los datos persistentes se utilizarán mecanismos de replicación, respaldo y recuperación, complementados con una estrategia de recuperación ante desastres.

Esta arquitectura permite combinar alta disponibilidad, tolerancia a fallos, escalabilidad horizontal, auto-healing y recuperación ante desastres, manteniendo la plataforma preparada para futuras ampliaciones.

3.2.11. Monitoreo y observabilidad

Para implementar la observabilidad de la plataforma se selecciona una arquitectura basada en **OpenTelemetry + ELK Stack (Elasticsearch, Logstash y Kibana)**.

OpenTelemetry se utilizará para instrumentar los servicios y recopilar **trazas, métricas y logs**. El **OpenTelemetry Collector** permitirá recibir, procesar y exportar esta información hacia la plataforma de observabilidad.

El **ELK Stack** se utilizará como plataforma para procesar, almacenar, consultar y visualizar la información recopilada. **Elasticsearch** será utilizado para el almacenamiento e indexación, **Logstash** para el procesamiento y transformación de los datos, y **Kibana** para su visualización y análisis.

Justificación

1. Trazabilidad distribuida de las operaciones

OpenTelemetry permite generar y propagar trazas distribuidas entre los diferentes componentes de la plataforma.

Una operación realizada por un cliente puede atravesar el **API Gateway**, diferentes microservicios, Kafka, Redis y sistemas externos. Las trazas permiten relacionar estas operaciones y determinar en qué componente se produjo un error o una demora.

Esto facilita la investigación de incidentes en una arquitectura distribuida.

2. Separación entre instrumentación y plataforma de análisis

OpenTelemetry es un estándar abierto y neutral respecto al proveedor. Los microservicios pueden generar y exportar telemetría sin quedar directamente vinculados a una plataforma comercial específica.

El **OpenTelemetry Collector** permite centralizar la recepción y procesamiento de la telemetría antes de enviarla al stack de observabilidad.

Esta separación permite modificar posteriormente los componentes utilizados para almacenar o visualizar la información sin tener que modificar la instrumentación de todos los servicios.

3. Centralización de logs, métricas y trazas

El ELK Stack permite centralizar grandes cantidades de información para facilitar su búsqueda, consulta y análisis.

Elasticsearch proporciona almacenamiento e indexación, **Logstash** permite procesar y transformar los datos, mientras que **Kibana** proporciona dashboards y herramientas de visualización.

La centralización permite correlacionar información proveniente de diferentes componentes durante la investigación de un incidente.

4. Integración con la arquitectura de microservicios

La solución puede integrarse con los servicios desarrollados con **Java y Spring Boot**, además de los componentes desplegados sobre Kubernetes.

De esta manera, se puede obtener información de observabilidad del API Gateway, microservicios, Kafka, Redis y otros componentes relevantes de la plataforma.

Alternativa evaluada: Dynatrace

Como alternativa se evaluó **Dynatrace**, una plataforma comercial de observabilidad que integra capacidades de monitoreo de aplicaciones, métricas, logs y trazas.

Una de sus principales ventajas es proporcionar una solución integrada para supervisar aplicaciones distribuidas y facilitar la investigación de incidentes, reduciendo la cantidad de componentes que deben administrarse.

Sin embargo, Dynatrace implica una dependencia de una plataforma comercial y de su modelo de licenciamiento.

La combinación de **OpenTelemetry + ELK** proporciona mayor control sobre los componentes de la solución y permite utilizar tecnologías abiertas para la recopilación, almacenamiento y visualización de la información.

Por estos motivos, se selecciona **OpenTelemetry + ELK Stack** como solución de observabilidad.

Aplicación en la arquitectura

La solución se organizará de la siguiente manera:

Microservicios / API Gateway / infraestructura → OpenTelemetry → OpenTelemetry Collector → ELK Stack

Dentro del ELK Stack:

OpenTelemetry Collector → Logstash → Elasticsearch → Kibana

La información podrá analizarse mediante:

- **Traces:** permiten seguir el recorrido de una operación entre los diferentes servicios.
- **Logs:** permiten identificar errores, excepciones y eventos generados por las aplicaciones.
- **Metrics:** permiten analizar indicadores de rendimiento, disponibilidad y utilización de recursos.
- **Kibana:** permite visualizar y correlacionar la información mediante dashboards y consultas.

Ante un incidente, la combinación de estas señales permitirá seguir el recorrido de la operación, identificar el componente donde se produjo el problema y analizar los registros y métricas relacionados.

Esta arquitectura proporciona una solución de observabilidad **desacoplada, extensible y basada en tecnologías abiertas**, adecuada para una plataforma financiera distribuida mediante microservicios.

3.2.12. Seguridad y cumplimiento normativo

Para proteger la información y las operaciones de la plataforma se propone aplicar una arquitectura de seguridad basada en múltiples capas, utilizando autenticación y autorización centralizadas, comunicaciones cifradas, gestión segura de secretos, control de acceso y monitoreo de eventos de seguridad.

La seguridad se aplicará tanto a las aplicaciones utilizadas por los clientes como a las comunicaciones entre los diferentes servicios internos y sistemas externos.

Justificación

1. Comunicaciones protegidas mediante TLS

Las comunicaciones entre clientes, API Gateway, microservicios y sistemas externos se realizan mediante HTTPS utilizando TLS.

Esto permite proteger la información durante su transmisión y reducir el riesgo de interceptación o modificación de los datos.

Este mecanismo será utilizado especialmente en operaciones sensibles como autenticación, consultas de información financiera, transferencias y comunicación con sistemas externos.

2. Autenticación y autorización centralizadas

La autenticación de los usuarios se realizará mediante Keycloak utilizando OAuth 2.0 y OpenID Connect, mientras que los servicios validará los tokens de acceso antes de permitir la ejecución de operaciones.

La autorización permitirá determinar qué operaciones puede realizar cada usuario y evitará que un usuario autenticado pueda acceder automáticamente a cualquier recurso de la plataforma.

Esta estrategia permite centralizar la administración de identidades y aplicar políticas de acceso de manera consistente.

3. Gestión segura de secretos

Las credenciales, claves, tokens y demás información utilizada para conectarse con bases de datos o servicios externos no deberán almacenarse directamente en el código fuente ni en las imágenes de los contenedores.

Se utilizará un mecanismo de gestión de secretos proporcionado por la infraestructura cloud para almacenar y controlar el acceso a esta información.

De esta manera, los secretos pueden administrarse independientemente del código de las aplicaciones y pueden ser rotados sin necesidad de modificar el código fuente.

Cumplimiento normativo

La arquitectura debe considerar la legislación ecuatoriana aplicable a la protección de datos personales. La Ley Orgánica de Protección de Datos Personales establece principios y obligaciones relacionados con el tratamiento y seguridad de los datos personales. Además, la normativa identifica los datos biométricos como datos sensibles.

Esto resulta especialmente importante para el proceso de onboarding, debido a que el reconocimiento facial implica el tratamiento de información biométrica.

Por esta razón, el proceso de onboarding deberá aplicar controles específicos sobre la recopilación, tratamiento, almacenamiento, acceso y protección de esta información, de acuerdo con las obligaciones legales aplicables.

También se debe considerar la normativa de la Superintendencia de Bancos relacionada con seguridad de la información y ciberseguridad. Esta normativa contempla controles de acceso, gestión de incidentes, evaluación de riesgos, protección de la información y resiliencia tecnológica.

Cuando las operaciones o componentes de la entidad se encuentren dentro del alcance correspondiente, también deberán considerarse estándares específicos de la industria, como PCI DSS para los entornos relacionados con datos de tarjetas de pago. PCI DSS v4.0.1 incluye controles relacionados con protección de datos, control de acceso, monitoreo y pruebas de seguridad.

Aplicación en la arquitectura

La seguridad se aplicará mediante diferentes capas:

Cliente → HTTPS/TLS → API Gateway → Autenticación y autorización → Microservicios → Sistemas internos y externos

Los accesos y operaciones relevantes serán registrados mediante el Servicio de Auditoría, mientras que la información de observabilidad será procesada mediante OpenTelemetry y ELK.

De esta manera, la seguridad no depende de un único componente, sino que se distribuye entre autenticación, autorización, cifrado, control de acceso, gestión de secretos, auditoría, monitoreo y cumplimiento normativo.

3.2.13. Gestión de costos e infraestructura cloud

Para desplegar la plataforma se propone utilizar infraestructura cloud de AWS, aprovechando los servicios administrados necesarios para ejecutar los componentes de la arquitectura y reducir la carga operativa asociada a la administración de infraestructura.

La infraestructura se diseñará de forma que los recursos puedan escalar de acuerdo con la demanda, evitando mantener permanentemente una capacidad superior a la necesaria.

Justificación

1. Uso de servicios administrados

Se utilizarán servicios administrados de AWS cuando aporten beneficios en componentes que requieren alta disponibilidad, respaldo y mantenimiento de infraestructura.

Esto permite reducir tareas operativas como mantenimiento de servidores, aplicación de actualizaciones y administración de determinados componentes de infraestructura.

Por ejemplo, la ejecución de los microservicios puede gestionarse mediante Amazon EKS, mientras que los componentes persistentes pueden utilizar servicios administrados de bases de datos y almacenamiento.

2. Escalabilidad según la demanda

La arquitectura basada en contenedores permite incrementar o reducir la cantidad de instancias de los servicios según la carga existente.

Los servicios que reciban una mayor cantidad de solicitudes pueden escalar horizontalmente, mientras que aquellos con menor demanda pueden mantenerse con una cantidad menor de réplicas.

Esto permite adaptar el consumo de infraestructura a las necesidades reales de la plataforma.

3. Separación entre componentes críticos y no críticos

No todos los componentes de la plataforma requieren los mismos recursos.

Los servicios relacionados directamente con operaciones financieras y autenticación tendrán mayores requisitos de disponibilidad, mientras que otros componentes podrán utilizar recursos ajustados a su nivel de demanda.

Esta separación permite asignar recursos de infraestructura de acuerdo con la importancia y comportamiento de cada componente.

Alternativa evaluada: infraestructura propia

Como alternativa se consideró implementar la plataforma utilizando infraestructura física o servidores propios administrados directamente por la entidad.

Esta opción proporciona un mayor control sobre la infraestructura y puede ser conveniente cuando existe una inversión previa importante en centros de datos.

Sin embargo, requiere que la entidad asuma directamente los costos relacionados con adquisición de hardware, mantenimiento, renovación, capacidad adicional, disponibilidad y recuperación ante fallos.

La utilización de infraestructura cloud permite convertir una parte importante de estos costos en consumo de servicios y facilita aumentar o reducir la capacidad de acuerdo con las necesidades de la plataforma.

Por estos motivos, se selecciona una infraestructura cloud basada en AWS para el despliegue de la solución.

Estrategia de control de costos

Para controlar los costos de la plataforma se deberán establecer mecanismos de seguimiento del consumo de recursos.

Se deberán monitorear factores como:

- Consumo de CPU y memoria de los servicios.
- Cantidad de instancias utilizadas.
- Almacenamiento utilizado.
- Tráfico de red.
- Cantidad de mensajes procesados.
- Consumo de servicios administrados.
- Recursos utilizados por los entornos de desarrollo, pruebas y producción.

También se recomienda separar los ambientes de desarrollo, pruebas y producción para evitar mantener recursos de producción innecesarios en ambientes que no requieren disponibilidad permanente.

La capacidad de los servicios deberá revisarse periódicamente para ajustar los recursos asignados a la demanda real.

De esta manera, la infraestructura cloud permite combinar escalabilidad, disponibilidad y flexibilidad con una estrategia de control del consumo de recursos.

4. Conclusiones

La arquitectura propuesta para la plataforma financiera permite implementar una solución distribuida basada en microservicios, manteniendo separadas las responsabilidades de cada componente y facilitando su evolución futura.

Los diagramas C4 desarrollados permiten representar la solución desde diferentes niveles de detalle. El diagrama de contexto muestra las relaciones entre los usuarios, la plataforma financiera y los sistemas externos. El diagrama de contenedores presenta las aplicaciones, servicios, sistemas de datos y componentes de infraestructura. Finalmente, el diagrama de componentes muestra la distribución interna de responsabilidades y las principales interacciones entre los componentes.

La utilización de Spring Cloud Gateway permite centralizar el acceso a los servicios internos, mientras que Keycloak proporciona la gestión de identidad y autorización mediante OAuth 2.0 y OpenID Connect. Para el onboarding se incorpora un servicio independiente que coordina la verificación de identidad mediante reconocimiento facial.

La arquitectura utiliza Apache Kafka para desacoplar el procesamiento de las operaciones financieras de funcionalidades complementarias como auditoría y notificaciones. Para las notificaciones se plantea Latinia como proveedor principal y una API propia como mecanismo alternativo, permitiendo mantener la capacidad de notificar ante una indisponibilidad del proveedor principal.

El acceso a los sistemas existentes se realiza mediante componentes Adapter, evitando que los servicios de negocio dependan directamente de las interfaces particulares de la Plataforma Core, el Sistema Complementario o los sistemas externos.

Para mejorar el rendimiento de las consultas frecuentes se incorpora Redis como mecanismo de caché, manteniendo los sistemas existentes como fuentes principales de información.

La ejecución de los microservicios sobre Amazon EKS permite utilizar capacidades de escalabilidad horizontal, distribución entre zonas de disponibilidad y recuperación automática de contenedores. Esto se complementa con mecanismos de respaldo y recuperación para los componentes persistentes.

Para la observabilidad se seleccionó OpenTelemetry junto con ELK Stack, permitiendo recopilar, procesar, almacenar y visualizar trazas, métricas y logs. Esta información facilita la investigación de incidentes y permite seguir el recorrido de una operación a través de los diferentes componentes de la plataforma.

Finalmente, la arquitectura incorpora mecanismos de seguridad como comunicaciones cifradas, autenticación y autorización centralizadas, gestión de secretos, auditoría y controles de acceso. Estos elementos deberán complementarse con el cumplimiento de la legislación ecuatoriana y de las regulaciones y estándares aplicables a las entidades financieras.

En conjunto, la solución propuesta busca mantener un equilibrio entre seguridad, disponibilidad, escalabilidad, observabilidad, desacoplamiento y control de costos, proporcionando una base tecnológica que puede extenderse posteriormente con nuevos servicios y funcionalidades.